PhantomStream: DOM-Native Live Mirroring for Agentic Browsing

Lakshman Turlapati
Full Self Browsing
Email: lakshmanturlapati@gmail.com
Code: https://github.com/fullselfbrowsing/PhantomStream

***Abstract*: AI agents increasingly operate real browsers on a user's behalf, but the interfaces used to supervise them are usually pixel streams: screenshots, WebRTC screen capture, or Chrome DevTools Protocol screencasts. Pixels show what the browser looked like, but they do not preserve which DOM element the agent is about to touch, they spend bandwidth on unchanged frames, and they make intervention a coordinate problem. We present PhantomStream, a DOM-native live mirroring system that streams a browser tab as a sanitized snapshot plus display-matched MutationObserver diffs addressed by stable node identities. The viewer rebuilds the page in an inert sandbox, applies node-addressed diffs, mirrors side channels such as scroll and media state, and can map authorized remote-control input back to the real browser. PhantomStream ships as the public package `@full-self-browsing/phantom-stream` version 0.2.1 and is verified by 712 automated tests. In a deterministic local benchmark over 5 site-shaped pages and 3 activity levels, PhantomStream used a median 79.78 KB per run, 75.09 percent less than a CDP PNG screencast, while preserving full semantic addressability. The same benchmark shows a real tradeoff: rrweb's compact event stream used only 14.03 KB, less than PhantomStream, because rrweb is optimized for record/replay rather than live, sandboxed, remotely controllable mirroring. These results frame PhantomStream as a system for agent supervision: not the smallest DOM recorder, and not a video codec, but a low-latency mirror whose main artifact remains structured browser state.**

***Index Terms*: agent observability, DOM streaming, browser mirroring, co-browsing, remote control, session replay, MutationObserver**

I. Introduction

Browser agents need supervision. A human operator wants to see which page an agent is on, what content it is reading, which element it is about to click, and whether intervention is needed. The simplest interface is a pixel stream: capture the tab as screenshots, a WebRTC screen stream, or a CDP screencast. That choice is attractive because pixels are universal. It is also a poor abstraction for agent supervision. Pixels do not identify DOM nodes, do not reveal whether an input is safe to replay as a browser-native action, and do not become cheaper when the page is mostly idle. A still dashboard costs bandwidth every frame.

PhantomStream takes the opposite path. It mirrors the page as browser structure: a full snapshot, small DOM diffs, stable node identities, and side channels for state that MutationObserver alone cannot observe. The system began as the live preview inside Full Self Browsing and was extracted into a standalone framework. The extracted package preserves the production reliability mechanisms that made the original useful: display-cadenced mutation batching, session identity, watchdogs, relay caps, truncation recovery, capture-side masking, and a sandboxed renderer.

The main design claim is that a live browser mirror for agents should preserve three properties at once. It should be *live*: mutations must arrive quickly enough for a human to follow the agent. It should be *safe*: attacker controlled page markup must not become script in the viewer, and configured private content must not leave the source page. It should be *semantic*: the receiving side should still know which node is being highlighted, updated, or controlled. PhantomStream is an attempt to keep all three without building a pixel pipeline.

This paper makes four contributions. First, it describes a DOM-native live mirror architecture built from snapshots, node-addressed diffs, side channels, and an authorized reverse-control path. Second, it presents the security and reliability contract needed for such a mirror to be embedded safely. Third, it documents the engineering choices that separate PhantomStream from generic session replay, including WeakMap-backed identity, sidecar reconstruction for modern DOM features, and a parent-realm media player that leaves the mirror sandbox scriptless. Fourth, it reports a deterministic local benchmark against rrweb record/replay and CDP screencast baselines. The benchmark is deliberately modest: it is reproducible in this repository, and it avoids public-site or WebRTC claims until a larger replayable corpus exists.

II. Background and Related Work

A. Pixel remote display

Remote display systems such as VNC operate at the framebuffer level. The Remote Framebuffer protocol is intentionally independent of any particular window system and lets a client view and control another graphical interface [1]. Browser-facing pixel streams follow the same broad pattern. Web screen capture exposes a user's display or a portion of it as a media stream through `getDisplayMedia` [2]. Chrome DevTools Protocol offers `Page.startScreencast`, which starts sending compressed screenshot frames through `screencastFrame` events [3]. These baselines are excellent for visual universality. They are weak for semantic supervision: inside the stream, an input field and a paragraph are just pixels.

B. Session replay

rrweb records and replays web sessions by serializing DOM state and incremental events [4]. Its guide separates recorder and replayer deployment and frames the project around record/replay workflows [5]. PhantomStream shares the broad idea that DOM changes can be transported more cheaply than pixels. The goal differs. rrweb is optimized to store and replay sessions. PhantomStream is optimized to operate as a live mirror for an active browser agent: it keeps a current viewer, exposes node identity to hosts, emits transport health, preserves a reverse-control mapping, and treats privacy masking and sandboxing as part of the runtime contract rather than as only a post-processing concern.

C. Browser platform primitives

MutationObserver is the browser primitive that makes incremental DOM streaming practical [6]. It observes structural and attribute changes but does not cover every state that matters to a mirror, such as form value property drift or media playback. PhantomStream therefore uses MutationObserver as one input, not as the whole protocol. It adds value diffs, media state, scroll, overlays, dialog events, and subtree recovery. On the viewer side, Content Security Policy controls which resources a document may fetch or execute [7]; PhantomStream combines a strict CSP with an iframe sandbox whose token list is exactly `allow-same-origin`, never `allow-scripts`.

III. System Design

A. Design requirements

PhantomStream's design starts from the constraints of an agent supervision surface rather than an archive format. The stream must be readable immediately after attachment, which requires a full baseline snapshot before incremental events. It must remain cheap while a page is idle, which rules out an unconditional frame clock. It must be safe to embed in a separate operator interface, which means serialized page content is untrusted at both capture and render time. It must be actionable, because supervision often ends with intervention: the host needs a way to resolve a mirrored node back into the geometry and identity of the controlled page.

The resulting lifecycle is a small state machine, shown in Figure 1. A host injects capture, capture sends a sanitized snapshot, the viewer becomes live after indexing that snapshot, diffs and side channels keep the mirror current, and recovery paths can request either a fresh snapshot or a bounded subtree. Remote control is deliberately outside the mirror document: the viewer exposes mappings, but the host decides whether an input is authorized and how it should be dispatched to the browser.

Figure 2 shows the end-to-end flow. Capture runs in the controlled page, emits a snapshot and diffs through an injected transport, the relay fans out raw frames under a one MiB cap, and the viewer reconstructs the page in an inert iframe.

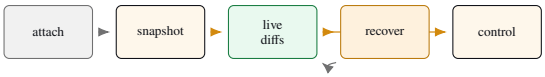

Fig. 1. Runtime lifecycle. A stream becomes useful after the first snapshot; diffs are the normal path, while re-snapshot and subtree requests repair drift or budgeted omissions.

B. Snapshot and diff stream

At stream start, capture clones the page body, walks the live and cloned trees in parallel, and emits sanitized HTML plus sidecar metadata. Default mode inlines a curated set of roughly 85 visual CSS properties. This is a production-driven choice: full computed-style enumeration over hundreds of properties made heavy pages unusably slow, while the curated list preserves common visual fidelity. Opt-in CSSOM mode transports scoped stylesheet sources and live stylesheet ops for hosts that prefer stylesheet-centric capture.

After the snapshot, a MutationObserver batches structural, attribute, and text changes on requestAnimationFrame so delivery follows the page's paint cadence. Diff ops are addressed by opaque node ids. Added subtrees carry their own serialized HTML and preorder `nodeIds` sidecars so the renderer can index new mirror nodes without selector lookup.

C. Protocol frames and recovery

The wire protocol is intentionally small. The dominant frame is the snapshot: sanitized body HTML, viewport dimensions, scroll position, shell attributes, sidecar node ids, open shadow-root payloads, accessible frame payloads, style sources, and snapshot counters. Incremental mutation frames carry add, remove, text, attribute, reorder, value, media, and frame-refresh operations. Independent side channels carry scroll, overlay, dialog, media synchronization, media hints, and subtree responses. Each frame includes enough stream identity for the renderer to reject cross-generation updates once a fresh snapshot has replaced the previous view.

Recovery is part of the normal protocol, not an exceptional control path. If a snapshot would exceed the relay budget, capture drops whole offscreen subtrees above a viewport-relative threshold and records placeholders with node ids. The viewer can later request one of those subtrees when the user scrolls, focuses a region, or asks the host to inspect a missing area. If the renderer detects that diff application has drifted too far from its indexed tree, it latches a re-snapshot request instead of continuing to apply diffs against uncertain state. This keeps the renderer simple: every recovery path returns to the same snapshot-plus-index invariant.

D. Stable identity without page mutation

The original FSB design stamped `data-fsb-nid` attributes onto the observed page. The standalone package removes that mutation. Capture owns identity in a `WeakMap<Element,string>` and emits ids only as wire sidecars. The renderer builds a private `Map` from ids to mirror nodes after sanitization. This keeps page-owned attributes ordinary page data and still lets overlays, subtree requests,

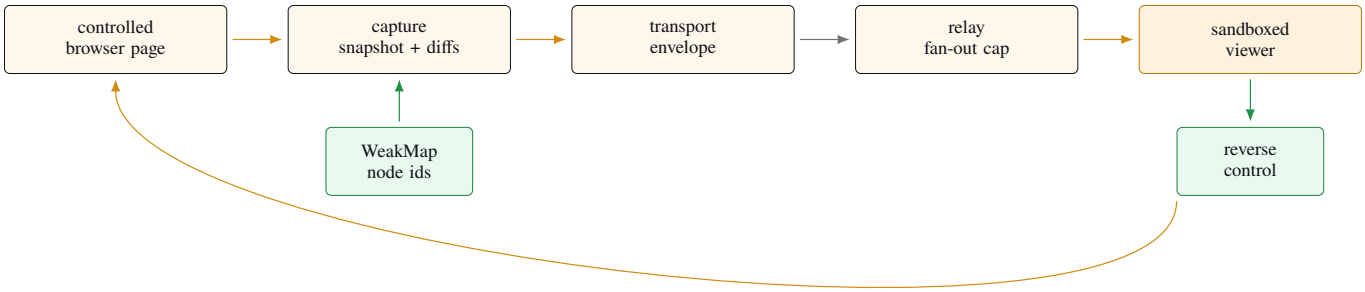

Fig. 2. PhantomStream mirrors browser state rather than pixels. Capture emits sanitized snapshot and diff frames addressed by stable node ids; the viewer reconstructs the mirror in a sandbox and can map authorized input back to the controlled page.

value diffs, media state, and remote control address the same node.

E. Modern DOM and media

The mirror handles several browser features that ordinary mutation streams miss. Open shadow roots serialize as `shadowRoots[]` sidecars tied to host ids. Same-origin iframes serialize as scoped `frames[]` payloads, while cross-origin frames remain content-free placeholders. Form control values emit property diffs because many value changes do not appear as DOM mutations.

Media is mirrored by reference. Image, video, and audio URLs travel as URL strings and small playback-state messages; the relay never carries media bytes. Progressive media uses a `STREAM.MEDIA` side channel and a pure drift reconciler. Adaptive HLS or DASH can be driven from the parent realm through an optional lazy player and manifest hints. Player code never runs inside the sandboxed mirror iframe.

F. Remote control contract

Remote control is modeled as a separate protocol layered over the mirror. The viewer can resolve a node id to a host-document rectangle, highlight it, and return the active stream and snapshot identity associated with that rectangle. It can also translate pointer coordinates through the scale-to-fit mapping of the iframe. It does not directly mutate the controlled page. Instead, a host adapter validates and normalizes control frames, applies an authorization state machine, and dispatches browser-native actions only when control is active.

This separation is important for agent supervision. It means a passive viewer can remain fully inert while still showing semantic targets. It also means the same mirror can be embedded in different execution environments. A Playwright host can dispatch through Playwright primitives, an extension host can dispatch through content-script events, and a bookmarklet host can restrict itself to local inspection. The protocol object that crosses the relay is a request, not an implicit grant of authority.

IV. Reliability and Security

The production lesson behind PhantomStream is that a browser mirror fails in small ways before it fails completely. A mutation queue can stall, a service worker can be evicted, a page can exceed a relay cap, a start request can race module load, and a late diff from a previous session can arrive after a new page has loaded. The system makes these failure modes explicit. A content-side watchdog force-flushes stale queues. An MV3 alarm watchdog can request a fresh snapshot after service-worker eviction. Every stream frame carries session and snapshot identity; stale frames are rejected. Snapshots are budgeted under the relay cap by dropping whole offscreen subtrees after a single layout-read pass, and dropped subtrees can be requested later by node id.

Security is a two-sided pipeline. Capture routes every serialization path through a named `sanitizeForWire` chokepoint before transport. Render routes add-op HTML, shadow roots, frame payloads, subtree responses, attributes, and CSSOM text through render-side sanitizers before DOM insertion. The viewer document carries a CSP meta policy and is hosted in an iframe whose sandbox token is exactly `allow-same-origin`. The absence of `allow-scripts` is load bearing: mirrored markup can render, but script in the mirror cannot execute.

Privacy masking is capture-side. Password values are always masked; hosts can mask inputs, text regions, whole blocked subtrees, and asset or media URLs. Because v2.0 mirrors assets by URL reference, signed CDN URLs and playback state can themselves be private. The package therefore includes URL token stripping, custom URL redaction, media selectors that omit URL and timeline state, a fail-closed viewer fetch policy, and a document-level no-referrer posture.

The relay is intentionally raw. It admits clients to rooms, classifies frame sizes for diagnostics, enforces the per-message cap, and fans frames out without understanding DOM semantics. This avoids making the relay a privileged parser for untrusted page HTML. Endpoints own envelopes, compression, sanitization, and authorization decisions. The tradeoff is that a malicious or buggy endpoint can still send

TABLE I
MAIN RUNTIME RISKS AND CURRENT MITIGATIONS.

Risk	Mitigation
Script execution in viewer	Render-side sanitizer, CSP, and an iframe sandbox without `allow-scripts`.
Private content leakage	Capture-side masking for passwords, selectors, text, assets, and media references.
Stale or crossed frames	Stream session and snapshot identity on live frames.
Relay budget exhaustion	One MiB relay cap, snapshot budget, offscreen subtree truncation, and on-demand subtree recovery.
Unauthorized control	Host-owned authorization state plus normalized remote-control messages.

bad application payloads; the viewer must therefore retain its own guards even when the relay has accepted a frame.

V. EVALUATION

A. Questions and methodology

The benchmark asks three questions. First, how many bytes does each local mirror representation produce. Second, how quickly does a post-action update become observable. Third, what semantic information remains available to a receiver. The benchmark is not a public-web study. It is a deterministic local corpus designed to be rerun in the repository before the paper is rebuilt.

The corpus contains five pages: a static article, an infinite feed, a dashboard, a modern-fidelity page with shadow DOM and a same-origin iframe, and a media page with referenced video and audio. Each page runs three activities: idle, reading and scrolling, and an agent-like mutation sequence. This produces 15 measured runs. The systems are PhantomStream, rrweb recorder plus replay, and CDP `Page.startScreencast` PNG frames. The harness records bytes, startup time, post-action latency, pixel-diff percentage against a final page screenshot, and a semantic score. The semantic score is a four-point rubric: DOM structure, incremental DOM changes, stable node identity, and reverse-control targetability. PhantomStream scores four of four, rrweb scores two of four, and CDP scores zero because it emits pixels only.

WebRTC screen capture is not claimed in this draft. Headless local Chromium does not provide the same interactive `getDisplayMedia` permission surface as a real user-driven capture session, so the harness records that baseline as unavailable instead of substituting invented numbers.

B. Harness implementation

The benchmark harness starts a local HTTP server that serves only the deterministic corpus, package source modules, and tiny local media placeholders. Each scenario opens a fresh Playwright Chromium page at a fixed 1280 by 720 viewport. The activity function runs inside the page and returns after deterministic scrolls, input updates, or DOM insertions. The harness then waits a short settling window and collects system-specific measurements.

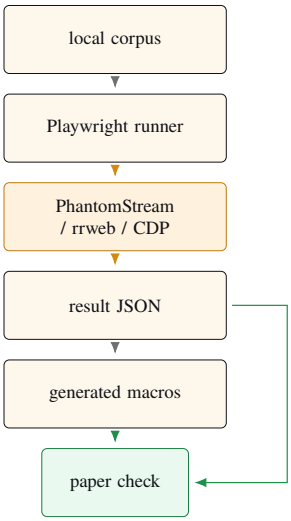

Fig. 3. Benchmark-to-paper pipeline. The paper imports generated result macros; the validator fails if required metrics are missing, non-finite, or mismatched.

TABLE II
DETERMINISTIC LOCAL BENCHMARK CORPUS.

Page	Activities	Purpose
article	idle, reading, agent	static text and scroll pressure
feed	idle, reading, agent	appended cards and list growth
dashboard	idle, reading, agent	tables, cards, and input changes
fidelity	idle, reading, agent	shadow DOM, iframe, forms
media	idle, reading, agent	referenced video and audio URLs

PhantomStream is measured by injecting `createCapture` from the local source tree and counting the UTF-8 bytes of every transport message. rrweb is measured by injecting its browser bundle and counting serialized recorder events; the harness also replays those events to obtain a final screenshot for the pixel-diff metric. CDP is measured by enabling `Page.startScreencast` and summing the decoded PNG frame bytes delivered by `screencastFrame` events. For all systems, startup time is the delay to the first useful frame or event, and post-action latency is the delay from the activity start timestamp to the first subsequent update.

The output contract is deliberately strict. The JSON result file records the environment, package version, corpus, activity list, baseline availability, per-scenario metrics, summary medians, and project metrics. A generated TeX file contains only macros derived from that JSON. The validator checks that all expected systems produced results, WebRTC is explicitly marked unavailable, no numeric metric is `NaN`, and the paper imports the generated macros rather than copying numbers by hand.

C. Bytes and latency

Figure 4 reports median bytes per run. PhantomStream's median was 79.78 KB. CDP screencast's median was 320.28

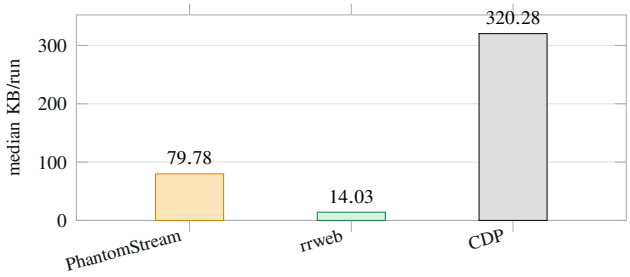

Fig. 4. Median bytes per deterministic local run. PhantomStream is smaller than CDP screencast but larger than rrweb's compact recorder stream.

Fig. 5. Median post-action latency on non-idle local activities.

KB, so PhantomStream used 75.09 percent fewer bytes in this local corpus. rrweb's median was only 14.03 KB, substantially smaller than PhantomStream. This is the clearest tradeoff in the data. PhantomStream's snapshot carries inline style and live-control metadata; rrweb's recorder emits a more compact replay log.

Figure 5 reports median post-action latency over non-idle activities. PhantomStream's median was 0.85 ms, close to rrweb's 0.8 ms and below CDP's 10.88 ms. These numbers are local headless measurements, not WAN latency claims. They do show that the DOM stream can react in the same order of magnitude as a recorder and earlier than the first post-action screencast frame in this setup.

D. Fidelity and semantics

The pixel-diff result is intentionally simple. CDP frames and PhantomStream final snapshots both reached a median 0 percent pixel-diff against the final page screenshot in this local corpus. rrweb replay had a median 8.781 percent pixel-diff, mostly from replay viewport and timing differences in the headless harness. The semantic score separates the systems more sharply: PhantomStream preserves all four rubric dimensions, rrweb preserves DOM structure and incremental change but not public live targetability, and CDP preserves none.

E. Implementation and assurance

PhantomStream version 0.2.1 is a source-first ESM package with one runtime dependency, `ws`. The implementation-facing JavaScript surface in `src`, `bin`, and `examples` contains 36 files and about 21379 lines. The test suite currently passes 712 tests

TABLE III
MEDIAN LOCAL FIDELITY AND SEMANTIC SCORES.

System	pixel diff (%)	semantic score
PhantomStream	0	1
rrweb	8.781	0.5
CDP screencast	0	0

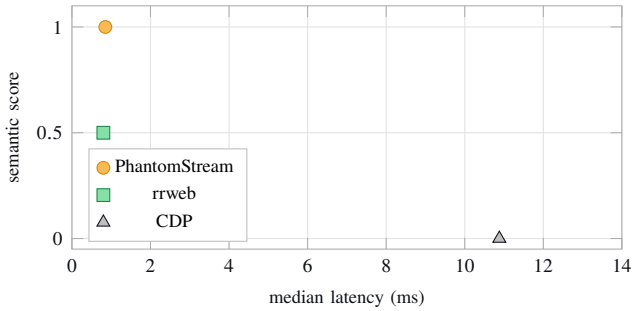

Fig. 6. Latency and semantic addressability tradeoff in the local benchmark. Lower-left is fast but less actionable; upper-left is the supervision target.

across 66 test files. The most important tests are not only unit tests; they include a differential oracle that runs reference and extracted captures over frozen fixtures, checks all intentional divergences against a ledger, and fails loudly on undeclared drift.

The benchmark is not yet in the same category as the test suite. It is a draft research harness whose purpose is to prevent the paper from making unsupported claims. Its scenarios are small enough to run during manuscript work and varied enough to catch obvious regressions in snapshot size, diff flow, replay fidelity, and baseline availability. That is the right role for this draft: it turns the paper's charts into reproducible artifacts before the project invests in a much larger public corpus.

VI. Discussion

The benchmark makes PhantomStream's position clearer than the original outline did. Against CDP screencast, the DOM-native mirror has the expected bandwidth and latency advantage in local runs while preserving semantic control handles. Against rrweb, PhantomStream is not the byte winner. That is acceptable because the systems optimize different products. rrweb is a compact session recorder. PhantomStream is a live, sandboxed, remotely controllable mirror. It spends bytes on inline visual state, stable ids, side channels, and viewer safety.

This distinction matters for browser agents. An agent-observability surface needs to answer questions that a replay log or a frame stream does not answer by default: which element is highlighted, whether a remote click can be mapped back through the mirror, whether a late diff belongs to the current tab, whether a masked subtree leaked, and whether a viewer fetch is allowed. PhantomStream makes those concerns explicit protocol and security decisions.

Pixels still have a place. They are the right fallback for WebGL, DRM video, native browser UI, and other surfaces that intentionally do not expose useful DOM state. They also provide a human-verifiable last resort when a DOM mirror is wrong. PhantomStream should therefore be understood as the primary channel for structured web content, not as a claim that every browser pixel should be serialized as DOM. A robust agent console will likely combine DOM-native mirroring with selective pixel fallbacks for opaque regions.

The rrweb comparison is similarly nuanced. rrweb's compactness is a real result, not an artifact to explain away. If the product is offline replay, rrweb's representation is attractive. If the product is live supervision with targetable nodes, explicit authority boundaries, and viewer fetch controls, a larger message stream can be justified. The next paper revision should quantify this boundary with ablations: no inline style, CSSOM-only style, no side channels, and no control metadata.

VII. Limitations

This draft has three important limitations. First, the evaluation is local and deterministic. It is suitable for regression and for an honest first paper draft, but it is not yet a public-site study. The next corpus should use HAR record/replay over real sites and should run each baseline under identical network and browser conditions.

Second, WebRTC is not measured here. The browser API is permission mediated and interactive by design [2]; a headless-only substitute would be a weak claim. A later evaluation should run WebRTC from a real capture surface and report its encoder settings.

Third, PhantomStream does not mirror everything. Cross-origin iframe content, DRM/EME video, MSE media with no discoverable manifest, WebGL/canvas frame streams, Web Audio, and mobile/non-Chromium contexts remain outside the current contract or degrade to placeholders. These are mostly browser security and bandwidth boundaries, not accidental omissions.

VIII. Future Work

The next step is a full v2.1 evaluation harness: public HAR record/replay corpus, scripted activity levels, WebRTC and CDP pixel baselines, rrweb live-mode replay, style-capture ablations, and a failure taxonomy that combines perceptual and DOM semantic metrics. The paper should also grow a primary-source related-work pass on co-browsing systems, session replay, remote display protocols, and agent-observability viewers. On the system side, useful extensions include a cursor channel, opt-in budgeted canvas refresh, caret and selection mirroring, and an rrweb-format export bridge for storage interoperability.

IX. Conclusion

PhantomStream is a DOM-native live mirror for supervising browser agents. It streams a sanitized snapshot and small node-addressed diffs, reconstructs the page in a scriptless sandbox, preserves stable semantic handles, and supports authorized reverse control. The current package is public, tested by 712 automated tests, and measured by a deterministic local benchmark. The results show the intended tradeoff: PhantomStream is far smaller than local CDP screencast and keeps full semantic addressability, but it is larger than rrweb's compact recorder stream. That is the core argument. Agent supervision does not need only pixels, and it does not need only replay. It needs a live, safe, semantically addressable mirror of the browser the agent controls.

References

[1] T. Richardson and R. Ltd., "The remote framebuffer protocol," RFC 6143, Internet Engineering Task Force, 2011, https://datatracker.ietf.org/doc/html/rfc6143.

[2] World Wide Web Consortium, "Screen capture," W3C Technical Report, 2025, https://www.w3.org/TR/screen-capture/.

[3] Chrome DevTools Protocol, "Page domain," Online protocol documentation, 2026, https://chromedevtools.github.io/devtools-protocol/tot/Page/.

[4] rrweb project, "rrweb: record and replay the web," Software and documentation, 2026, https://github.com/rrweb-io/rrweb.

[5] ——, "rrweb guide," Online documentation, 2026, https://github.com/rrweb-io/rrweb/blob/master/guide.md.

[6] WHATWG, "DOM standard," Living Standard, 2026, https://dom.spec.whatwg.org/.

[7] World Wide Web Consortium, "Content security policy level 3," W3C Technical Report, 2026, https://www.w3.org/TR/CSP3/.